

DataFrame and Series Reference

To import the library, use: `import polars as pl`. For functions below, assume a `DataFrame` `df` and a `Series` `s`. `DataFrame` (table) consists of `Series` (columns).

Name	Output
<code>df.shape</code>	Return a tuple containing number of rows and columns
<code>df.height</code>	Return a number of rows in the <code>DataFrame</code>
<code>df.width</code>	Return a number of columns in the <code>DataFrame</code>
<code>df.columns</code>	Return a list of column names
<code>df.head(n)</code> / <code>df.tail(n)</code>	Return a DataFrame containing the first / last <code>n</code> rows
<code>df[rows, cols]</code>	Return a DataFrame , Series , or scalar depending on selection (for numerical slices: inclusive from left, exclusive from right)
<code>df.slice(offset, n)</code>	Return a DataFrame <code>n</code> rows starting at position <code>offset</code>
<code>df.schema</code>	Return a mapping of column name to data type
Column Operations	
<code>df[column]</code> / <code>df.get_column(column)</code>	Return a Series
<code>df.with_columns(expr)</code> <code>df.with_columns(s1, s2)</code>	Return a DataFrame with a new column produced by <code>expr</code> Return a DataFrame with new columns made of <code>Series</code> <code>s1</code> and <code>s2</code>
<code>df.select(column1, column2)</code> <code>df.select(expr)</code>	Return a DataFrame containing the columns labeled <code>column1</code> and <code>column2</code> Return a DataFrame with a column added or replaced by <code>expr</code>
Row Operations	
<code>df[i]</code> <code>df.row(i)</code>	Return a DataFrame with a single row at position <code>i</code> Return a tuple of a single row at position <code>i</code>
<code>df.filter(cond)</code>	Return a DataFrame keeping only rows where the boolean expression <code>cond</code> is true
Series Manipulation	
<code>s.sort(descending=False)</code>	Return a sorted Series
<code>s.value_counts(sort=False)</code>	Return a two-column DataFrame (value, count) with the number of times each unique value appears
<code>s.is_in(values)</code>	Return a boolean Series indicating if each element is in <code>values</code> ; as an expression, <code>pl.col(column).is_in(values)</code>
<code>s.unique()</code>	Return a Series of the unique values of <code>s</code>
<code>s.fill_null(value)</code>	Return a Series where null values are replaced by <code>value</code>
<code>s.is_null()</code>	Return a boolean Series identifying null values
<code>s.cast(dtype)</code>	Return a Series cast to the type <code>dtype</code> (e.g. <code>pl.Int64</code> , <code>pl.Float64</code> , <code>pl.String</code>); as an expression, <code>pl.col(column).cast(dtype)</code>
<code>s.arg_max()</code> / <code>s.arg_min()</code>	Return the int position of a max / min value
<code>s.to_numpy()</code> / <code>s.to_list()</code>	Return a Series converted to a NumPy array / a Python list
DataFrame Manipulation	
<code>df.drop(column1, column2, ...)</code>	Return a DataFrame without the columns named <code>columns</code>
<code>df.sort(by=column, descending=False)</code>	Return a sorted DataFrame
<code>df.rename({old: new})</code>	Return a DataFrame with columns renamed using a dictionary mapping old names to new names
<code>df.unique()</code>	Return a DataFrame with duplicate rows dropped
<code>df.null_count()</code>	Return a one-row DataFrame counting nulls in each column
<code>df.drop_nulls()</code>	Return a DataFrame without the rows containing any null
<code>df.fill_null(value)</code>	Return a DataFrame where null values are replaced by <code>value</code>
<code>df1.join(df2, on=column, how='inner')</code> <code>df1.join(df2, left_on=column1, right_on=column2, how='inner')</code>	Return a DataFrame joining <code>df1</code> and <code>df2</code> (how: 'inner', 'left', 'right', 'full', 'cross')
<code>df.pivot(on, index=index, values=values, aggregate_function=aggregate_function)</code> <code>df.group_by(column).agg(expr)</code>	Return a DataFrame pivot table where columns are unique values from <code>on</code> , rows are unique values from <code>index</code> , and cells are values from <code>values</code> combined with <code>aggregate_function</code> ('mean', 'sum', 'len', ...) Return a DataFrame with one row per group and one column per aggregation expression, e.g. <code>df.group_by('g').agg(pl.col('x').mean(), pl.len())</code>
<code>df.group_by(column).mean()</code>	Return a DataFrame containing the mean (also: <code>.min()</code> , <code>.max()</code> , <code>.sum()</code> , <code>.head(n)</code> , <code>.tail(n)</code> , <code>.len()</code>) of each group for every other column, excluding null values
<code>grouped.first()</code> / <code>.last()</code>	Return a DataFrame containing the first / last row of each group

Expressions

An expression describes a computation on columns and is evaluated inside `select`, `with_columns`, `filter`, or `agg`. Expressions can be stacked.

Name	Output
<code>pl.col(column)</code> <code>pl.col(column1, column2)</code>	Refer to one or more columns by name
<code>pl.all()</code>	Refer to every column; <code>pl.col(pl.Float64)</code> refers to every column of that type
<code>pl.lit(value)</code>	A constant literal value (a number, string, etc.) usable inside an expression
<code>expr.alias(name)</code>	Name the output column of an expression name
<code>expr.sum() / .mean() / .min()</code> <code>.max() / .std() / .median()</code>	Aggregate a column to a single value (ignoring nulls); inside <code>select</code> this Return a one-row DataFrame
<code>expr.count()</code>	Number of non-null values in the column
<code>pl.len()</code>	Number of rows (including nulls)
<code>expr.n_unique()</code>	Number of distinct values in the column
<code>expr.first() / expr.last()</code>	First / last value of the column
<code>pl.when(cond).then(a).otherwise(b)</code>	Element-wise conditional: <code>a</code> where <code>cond</code> is true, else <code>b</code> ; extra <code>.when().then()</code> pairs may be chained
<code>expr.map_elements(f, return_dtype)</code>	Apply Python function <code>f</code> to each element (slow; prefer built-in expressions)

String Methods

String methods are accessed through the `.str` namespace on an expression or Series, e.g. `df.with_columns(pl.col('name').str.to_lowercase())`.

Name	Output
<code>s.str.len_chars()</code>	Return a Series containing the length (number of characters) of each string; <code>len_bytes()</code> counts bytes
<code>s.str.slice(offset, length=None)</code>	Return a Series where each element is a substring starting at <code>offset</code> (may be negative) of the given length; equivalent to pandas <code>s.str[a:b]</code> with <code>length = b - a</code>
<code>s.str.to_lowercase()</code> <code>s.str.to_uppercase()</code>	Return a Series of lowercase / uppercase versions of each string
<code>s.str.replace(pat, repl, literal=False)</code> <code>s.str.replace_all(pat, repl, literal=False)</code>	Return a Series replacing the first / all occurrence(s) of substrings matching <code>pat</code> with <code>repl</code> . When <code>literal=False</code> , <code>pat</code> is a RegEx pattern (capture groups are referenced as <code>\${1}</code>); when <code>literal=True</code> , it is a literal string
<code>s.str.contains(pat, literal=False)</code>	Return a boolean Series indicating if a substring matching the regex <code>pat</code> is contained in each string; see also <code>starts_with</code> , <code>ends_with</code>
<code>s.str.extract(pat, group_index=1)</code>	Return a Series of the first substring matching capture group <code>group_index</code> of regex <code>pat</code> (null if no match)
<code>s.str.extract_all(pat)</code>	Return a Series of lists containing all matches of regex <code>pat</code>
<code>s.str.split(by)</code>	Return a Series of lists, splitting each string at the delimiter <code>by</code> (no default; use ' ' for whitespace); access elements with <code>.list.get(i)</code>
<code>s.str.strip_chars(chars=None)</code>	Return a Series with leading and trailing whitespace (or <code>chars</code>) removed
<code>s.str.to_integer()</code> <code>s.cast(pl.Float64)</code>	Parse numeric strings to integers / floats

Regular Expressions

Operator	Description	Operator	Description
.	Matches any character except \n	*	Matches preceding character/group zero or more times
\	Escapes metacharacters	?	Matches preceding character/group zero or one time
	Matches expression on either side; has lowest priority of any operator	+	Matches preceding character/group one or more times
\d, \w, \s	Predefined character groups for digits (0–9), alphanumerics (a–z, A–Z, 0–9, and underscore), and whitespace, respectively	^, \$	Match the beginning and end of the string/line, respectively
\D, \W, \S	Inverse sets of \d, \w, \s, respectively	()	Capturing group used to create a sub-expression
{m}	Matches preceding character/group exactly m times	[]	Character class used to match any specified character or range; e.g. [abcde] is equivalent to [a-e]
{m,n}	Matches preceding character/group at least m and at most n times. If m or n is omitted, the lower/upper bound is 0 / unbounded, respectively	[^]	Inverted character class; e.g. [^a-c] matches all characters except a, b, and c

Capture group example:

```
import re

lines = '169.237.46.168 - - [26/Jan/2014:10:47:58 -0800] "GET ... HTTP/1.1"'

re.findall(
    r'\[\d+\/(\w+)\]\d+:\d+:\d+ -\d+\]',
    lines
)
# returns ['Jan']
```

Function	Description
re.match(pattern, string)	Returns a match if zero or more characters at beginning of string matches pattern, else None
re.search(pattern, string)	Returns a match if zero or more characters anywhere in string matches pattern, else None
re.findall(pattern, string)	Returns a list of all non-overlapping matches of pattern in string (if none, returns empty list)
re.sub(pattern, repl, string)	Returns string after replacing all occurrences of pattern with repl

Visualization

import seaborn as sns

Seaborn accepts Polars DataFrames. For example, sns.histplot(data=df, x=column).

import matplotlib.pyplot as plt

Matplotlib functions take sequences only. Use df['x'], df['x'].to_numpy() or df['x'].to_list(). For example, plt.hist(df['x']).

Polars also ships a built-in Altair-based interface. For example, df.plot.scatter(x='a', y='b'), df.plot.line(...), df.plot.bar(...).

SQL

Strings in SQL should use single quotes. Column names that contain a space should use double quotes.

```
SELECT [DISTINCT]
  { * | expr [[AS] c_alias]
    {, expr [[AS] c_alias] ... } }
FROM tbleref {, tbleref}
[[INNER | LEFT] JOIN table_name
  ON qualification_list]
[WHERE search_condition]
[GROUP BY colname {, colname...}]
[HAVING search_condition]
[ORDER BY column_list]
[LIMIT number]
[OFFSET number of rows];
```

Syntax	Description
SELECT column_expression_list	List is comma-separated. Column expressions may include aggregation functions (MAX, FIRST, COUNT, AVG, etc). AS renames columns. DISTINCT selects only unique rows.
FROM s INNER JOIN t ON cond	Inner join tables s and t using cond to filter rows; the INNER keyword is optional.
FROM s LEFT JOIN t ON cond	Left outer join of tables s and t using cond to filter rows.
FROM s, t	Cross join of tables s and t: all pairs of a row from s and a row from t
WHERE a IN cons_list	Select rows for which the value in column a is among the values in a cons_list.
ORDER BY RANDOM() LIMIT n	Draw a simple random sample of n rows.
ORDER BY a, b DESC	Order by column a (ascending by default), then b (descending).
CASE WHEN pred THEN cons ELSE alt END	Evaluate to cons if pred is true and alt otherwise. Multiple WHEN/THEN pairs can be included, and ELSE is optional.
WHERE s.a LIKE 'p'	Match each entry in the column a of table s to the text pattern p. The wildcard % matches at least zero characters.
LIMIT number	Keep only the first number rows in the return result.
OFFSET number	Skip the first number rows in the return result.